Interference-Aware Data Management with DRAM and CXL Memory

Research progress and experimental findings

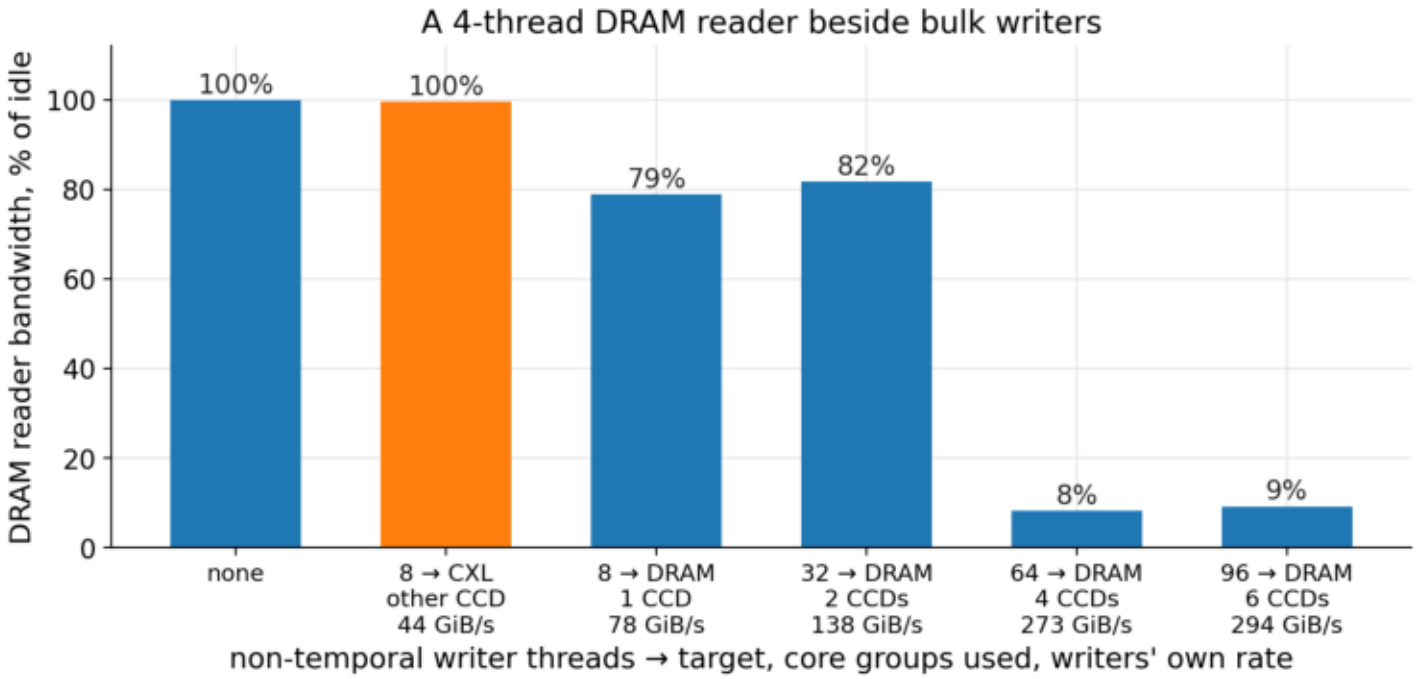
Progress report for Dr. Mueller · 2026-10-13 · c28 (AMD EPYC 9745, 4× CXL CZ120) and c35 (Optane)

Evidence tag on each slide: Hardware measurement · Real application · Trace-driven emulation · Trace simulation · Estimate · Proposal (not implemented)

Bulk writes into DRAM stall every DRAM reader; the same writes into CXL do not

Part I · Original research idea

Hardware measurement



Hardware measurement, c28 (EPYC 9745, NPS1): reader on CCD 0 (CXL case: CCD 4), writers on other CCDs; medians of 3–5 runs. 2026-09-14 / 2026-10-10

- Non-temporal writers on 4 of 8 core groups cut a 4-thread DRAM reader to 8% of its idle bandwidth. Writers on 1–2 groups cost it 18–21%. The loss is a cliff, not a slope.
- The memory controller enters write-drain mode when its write queue stays above the watermark. Reads then wait for short windows between write batches.
- Checkpoint staging and KV-cache stores are such bulk writes. Both grow with the model size.
- The same 8 writer threads into CXL cost the reader 0% when they run on another core group.

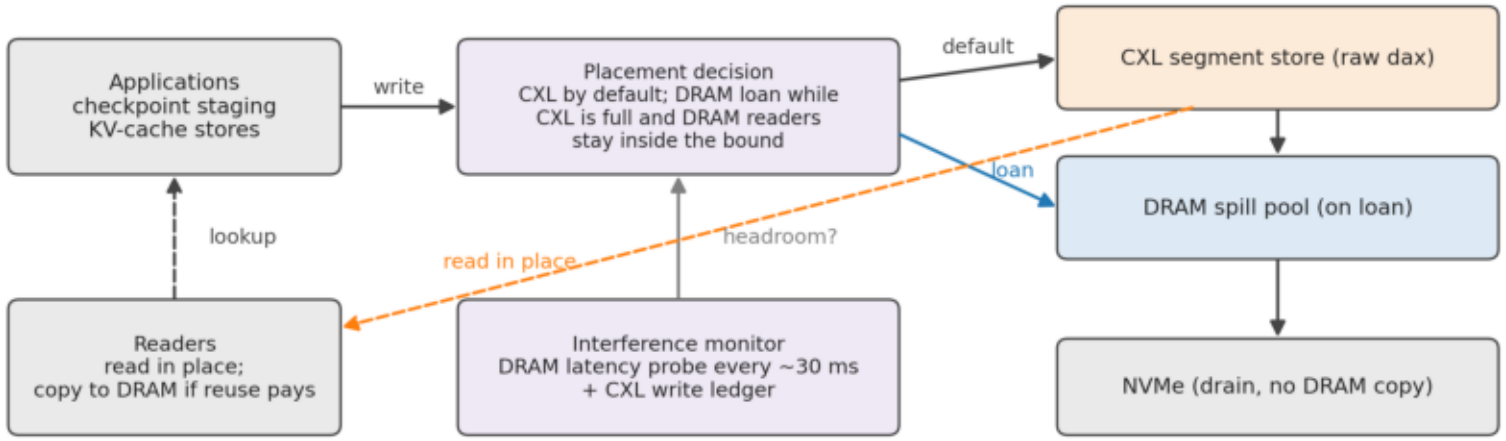
Takeaway: CXL is a second, independent memory path. Bulk writes can use it without stalling DRAM readers.

Data: 2026-09-08-dram-write-drain (run 2026-09-14) · 2026-10-09-cxl-topology-interference (2026-10-10) · figure f01_drain_cliff

Original design: write to CXL by default, borrow DRAM only with measured headroom

Part I · Original research idea

System built 2026-09-19



Built 2026-09-19 (cxlpath, user space). DRAM holds copies of CXL data (inclusive): DRAM adds speed, not capacity.

- Every bulk write goes to a segment store on raw CXL (Device-DAX). There is no DRAM staging copy and no system call on the data path.
- DRAM is lent to a writer only while CXL is full and a DRAM latency probe shows headroom for other readers. The loan is revoked at zero cost.
- Reads are served in place. A block is copied to DRAM only when its reuse pays for the copy.
- Checkpoints drain from CXL to NVMe without a DRAM copy.

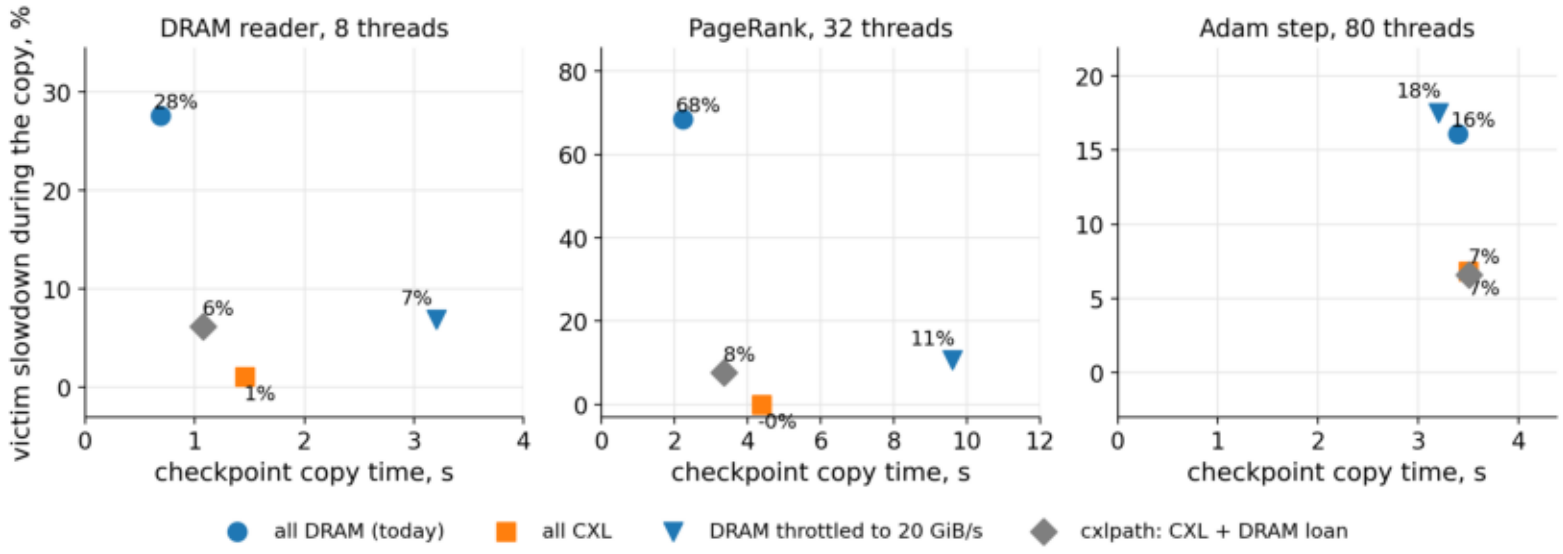
Takeaway: DRAM held copies of CXL data (inclusive placement). DRAM added speed, not capacity.

Data: docs/DESIGN.md §10 · system/cxlpath (user-space runtime, built 2026-09-19) · figure f03_arch_original

First results: the loan works beside moderate victims; beside saturated ones the guard lends nothing

Part I · Original research idea

Real application



Real applications on c28, 64 GiB checkpoint every 25 s (PageRank: 192 GiB / 30 s); medians of 3 runs. 2026-09-22/23

- 64 GiB checkpoints every 25 s beside real programs on c28; 3 runs each, medians.
- DRAM staging slows the reader 28% and PageRank 68%. All-CXL harms them ≤ 1% but copies 1.5–2× slower.
- cxlpath lends 23–34% of the chunks: 6–8% harm, copies 1.3–1.5× faster than all-CXL.
- Beside the 80-thread Adam step the guard refuses every loan: cxlpath equals all-CXL (7% harm, 3.5 s).

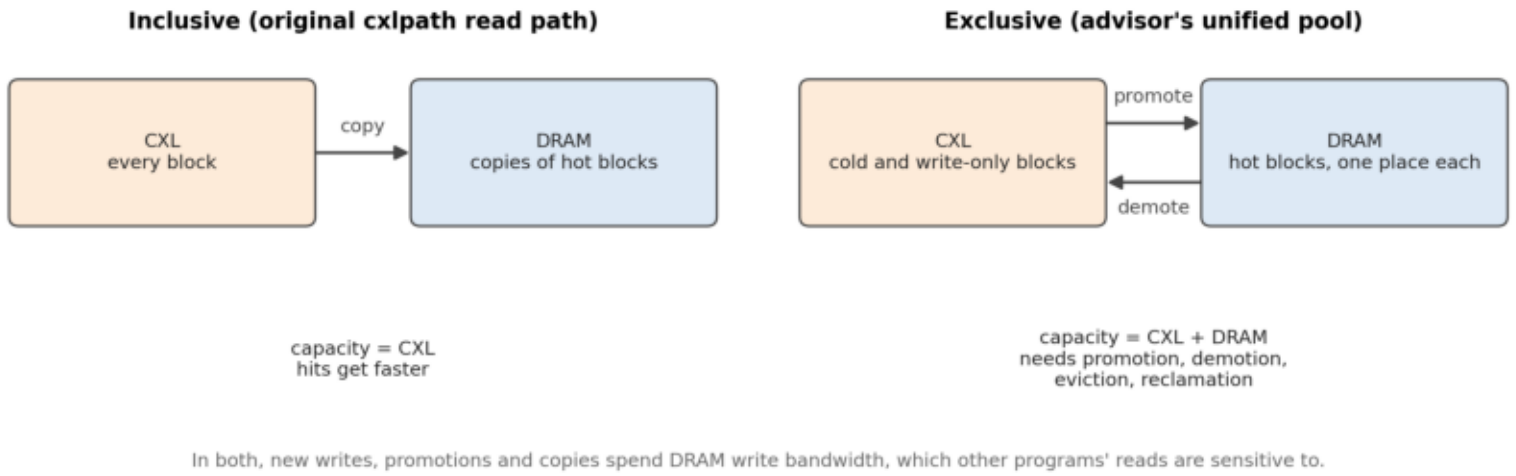
Takeaway: The guard decides the regime. Beside DRAM-heavy programs it finds no headroom. We meet this limit again in Part VI.

Data: 2026-09-22-three-policies-realdata/summary.csv (117 runs, 2026-09-22/23) · figure f02_original_core

The advisor's suggestion: manage DRAM and CXL as one pool with exclusive placement

Part II · Advisor's suggestion

Proposal (not implemented)



- Our read path kept copies of CXL blocks in DRAM. The cache capacity stayed equal to CXL.
- Proposal (2026-10-09): one manager for both tiers. Each block lives in one tier. The manager does allocation, write placement, promotion, demotion, eviction and reclamation.
- Exclusive placement adds DRAM's capacity to the cache. It also needs migration, and every migration is a DRAM write.

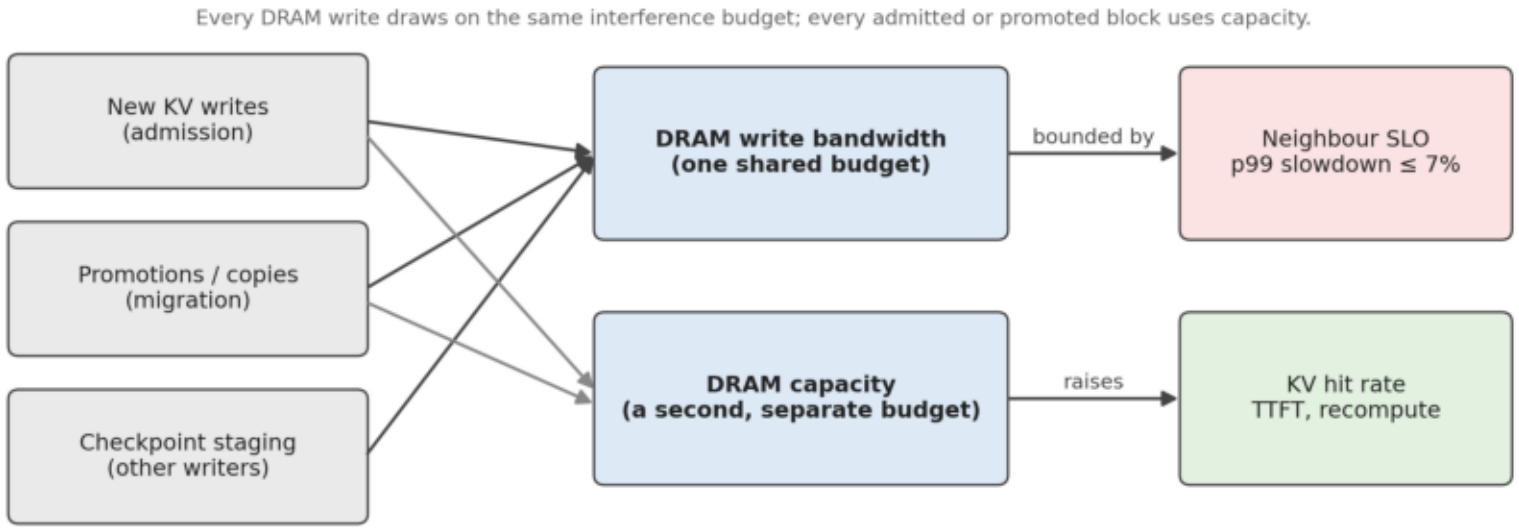
Takeaway: Decision: test the premises first. Build the full manager only if the data show that migration earns it.

Data: WORKLOG.md session 29 (discussion of 2026-10-09) · figure f04_unified_pool

Why unified management may matter: capacity and write bandwidth are two separate budgets

Part II · Advisor's suggestion

Analysis | Trace simulation



- DRAM can add effective KV capacity: +3 to +15 points of reuse hits next to 512 GiB of CXL in the trace simulation (Part V).
- DRAM writes spend the bandwidth that DRAM readers are sensitive to (Part I).
- New writes, promotions and checkpoint copies all draw on that one bandwidth budget. Admitted and promoted blocks draw on capacity.
- Placement and migration must respect both budgets at the same time.

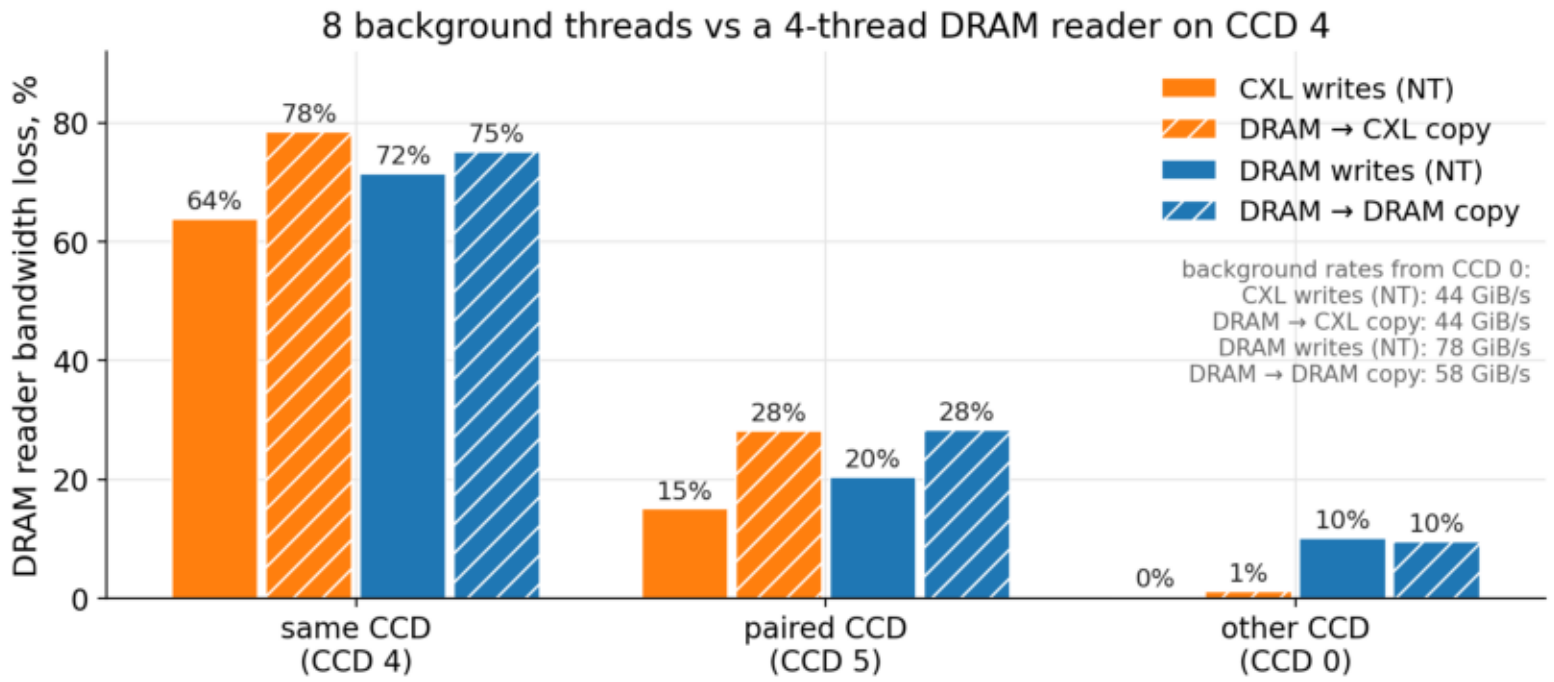
Takeaway: We keep the write-interference motivation and add capacity as a second resource. Nothing on this slide is a built mechanism.

Data: Decisions of 2026-10-09 · 2026-10-09-kv-capacity-characterization · figure f21_two_resources

CXL writes spare a DRAM reader only from an unrelated core group

Part III · Hardware characterization

Hardware measurement



Hardware measurement, c28, 5 runs × 3 s, medians; reader alone 98.0 GiB/s. 2026-10-10

- 8 threads writing CXL from the reader's own core group cut it 64%; from the paired group 15%; from any other group 0%.
- DRAM writes from the same places cost 72%, 20% and 10%.
- On the reader's own group the CXL rate stays at 44 GiB/s from 2 threads on, but the loss grows from 29% to 64% with 8 threads: stalled CXL requests crowd out the reader (the MIKU effect).
- A DRAM → CXL copy at the full 44 GiB/s from another group costs 1%.

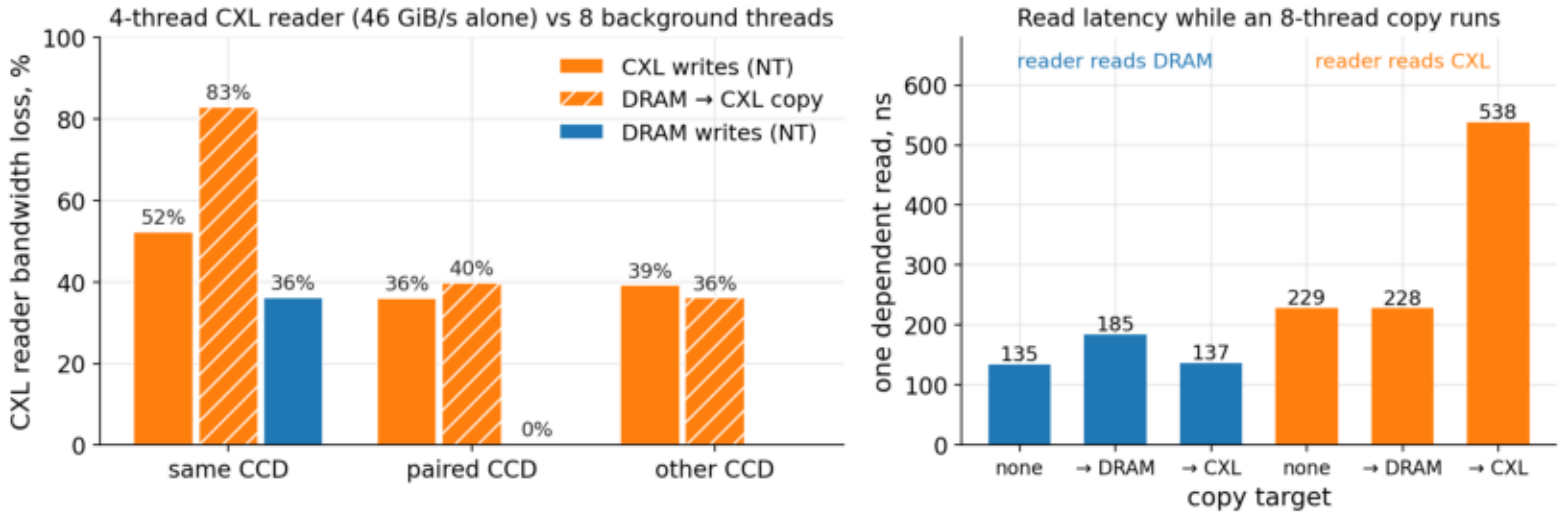
Takeaway: The premise holds under a placement condition. The system must put its copy threads on a group that runs no DRAM-sensitive tenant and is not paired with one.

Data: 2026-10-09-cxl-topology-interference/topo.txt (2026-10-10; 5 runs × 3 s, medians) · figure f05_topology

CXL readers lose 36–40% to CXL writes from anywhere; a dependent read takes 2.4× longer

Part III · Hardware characterization

Hardware measurement



Hardware measurement, c28; left: 2026-10-10 (medians of 5); right: 2026-10-06 (medians of 3), copy threads on CCDs 0–1

Takeaway: CXL bandwidth is a managed resource too. A bulk writer and a latency-sensitive reader on the same cards need coordination.

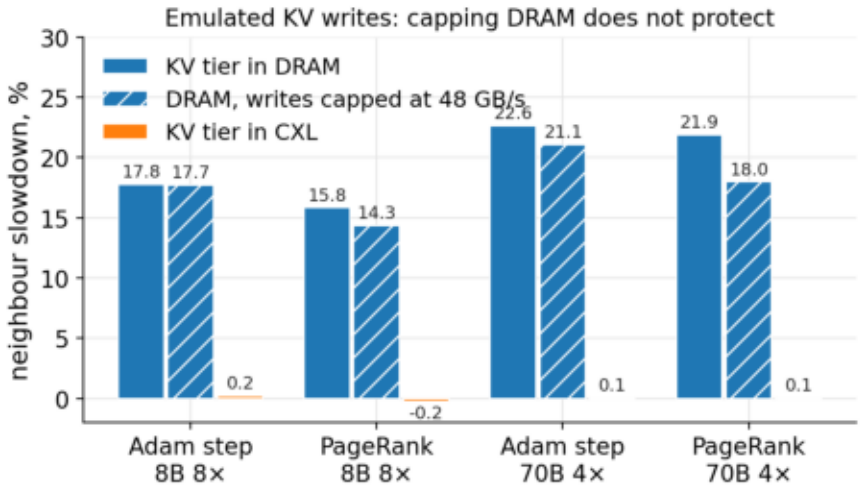
- A 4-thread CXL reader (46 GiB/s alone) keeps 60–64% of its bandwidth while 8 threads write CXL, from any core group.
- One dependent read from CXL: 229 ns alone, 538 ns during a copy into CXL. The copy itself slows from 44 to 30 GiB/s.
- DRAM writes do not touch a CXL reader unless they share its core group.
- A GPU reads DRAM and CXL at the same 12.5 GiB/s on this machine; its PCIe link is the limit.

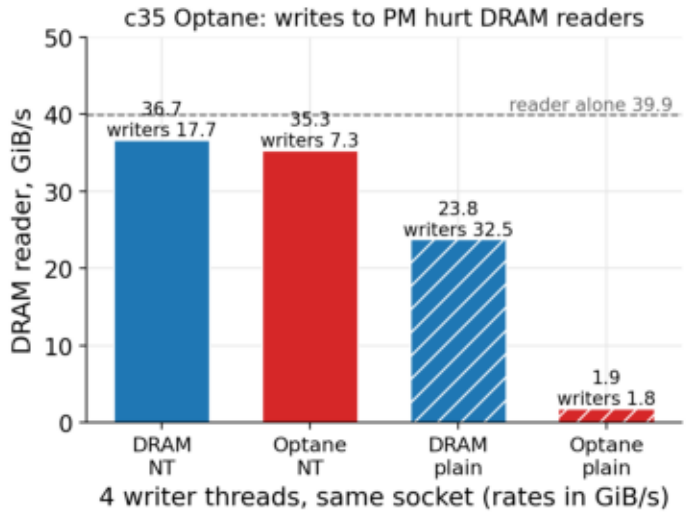
Data: 2026-10-09-cxl-topology-interference/topo.txt · 2026-10-07-read-cost/readcost.txt (medians of 3) · figure f06_cxl_reader

Throttling DRAM does not protect neighbours, and persistent memory is not a substitute for CXL

Part III · Hardware characterization

Trace-driven emulation | Hardware measurement





Left: host memory traffic emulation on c28, one run per cell (2026-10-10/11). Right: hardware measurement on c35, socket 1, medians of 3 (2026-10-11)

Takeaway: The protection comes from placement on an independent path. Persistent memory shares the DRAM controllers; CXL does not.

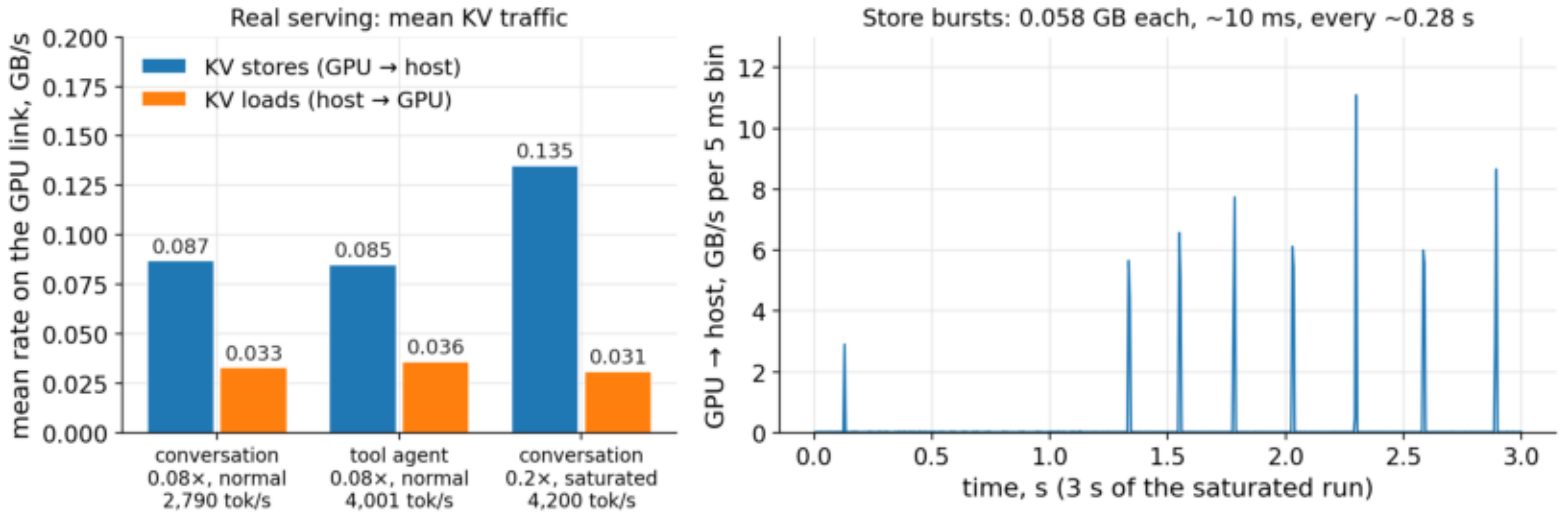
- DRAM writes capped at the CXL rate (48 GB/s) move the same bytes with the same bursts and still slow the Adam step 17.7% (uncapped: 17.8%). The CXL tier: 0.2%.
- Only a reader with spare bandwidth gains from the cap (6.2% → 3.8%). Busy neighbours react to the volume of DRAM writes, not to the peaks.
- On c35, non-temporal writes to Optane cost a same-socket DRAM reader 11.5% at 7.2 GiB/s; DRAM writes cost 8% at 17.8 GiB/s. Plain stores to Optane cut the reader from 39.9 to 1.9 GiB/s.

Data: 2026-10-11-throttled-dram-baseline · 2026-10-10-lmcache-kv-traffic/layer2 · 2026-10-06-optane-control (c35, run 2026-10-11) · figure f07_throttle_optane

Real KV serving on the RTX 4060 Ti writes ~0.1 GB/s in short bursts and disturbs nobody

Part IV · KV cache characterization

Real application



Real application: vLLM 0.31 + LMCache 0.5.5, Qwen2.5-1.5B on the RTX 4060 Ti (PCIe 4.0 ×8), c28, 2026-10-10/11; NVML cumulative counters, payload = counter / 1.12

Takeaway: One small GPU cannot produce interfering KV traffic. The 1× / 4× / 8× trace rates need server-scale traffic, which we emulate.

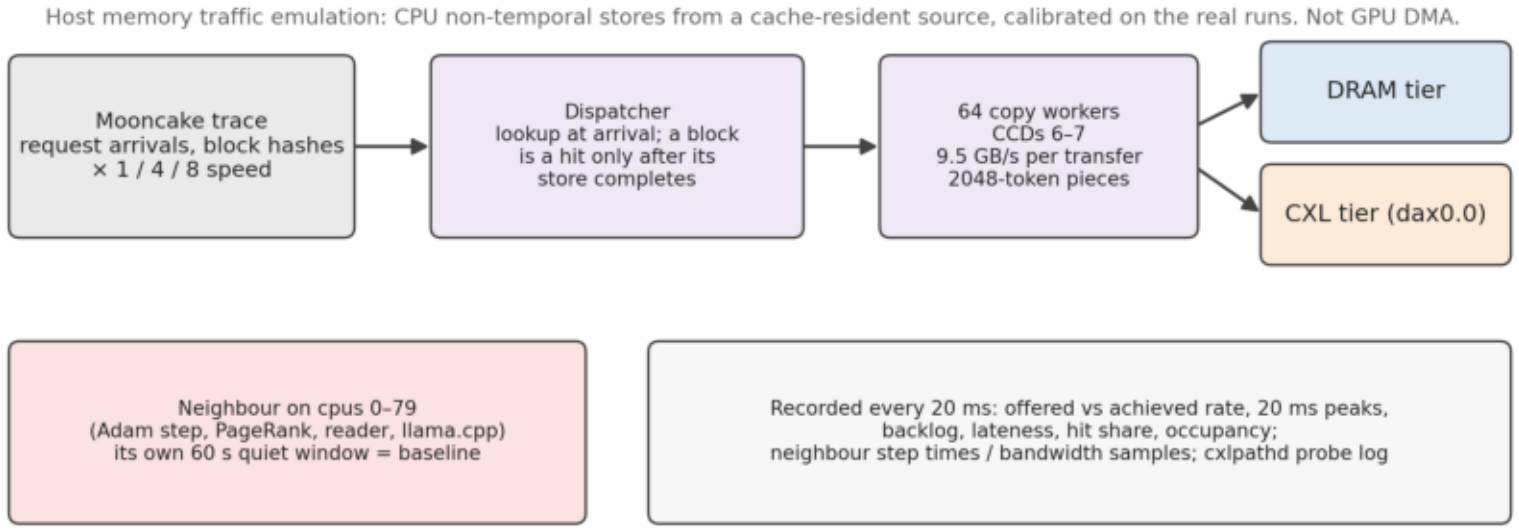
- vLLM 0.31 + LMCache 0.5.5, Qwen2.5-1.5B, RTX 4060 Ti 8 GB (PCIe 4.0 ×8). Mooncake traces replayed open-loop.
- The card sustains ~0.08× of the trace (2.8–4.0 k prompt tokens/s). At 0.2× it saturates: TTFT grows without bound.
- Each KV store is a 0.058 GB burst at ~9 GB/s lasting ~10 ms, every ~0.28 s. Mean traffic: 0.09–0.14 GB/s.
- Neighbours beside saturated serving: reader −0.07%, PageRank −0.15%, Adam +0.07%, llama.cpp +0.1% (drift-corrected).

Data: 2026-10-10-lmcache-kv-traffic/layer1 (2026-10-10/11; NVML cumulative PCIe counters, 5 ms bins) · figure f08_real_kv

Host memory traffic emulation reproduces server-scale KV traffic from the traces

Part IV · KV cache characterization

Trace-driven emulation



Takeaway: The emulation gives the traffic shape of a server, not a multi-GPU system. 70B 4× and 8× stand for several servers' aggregate traffic.

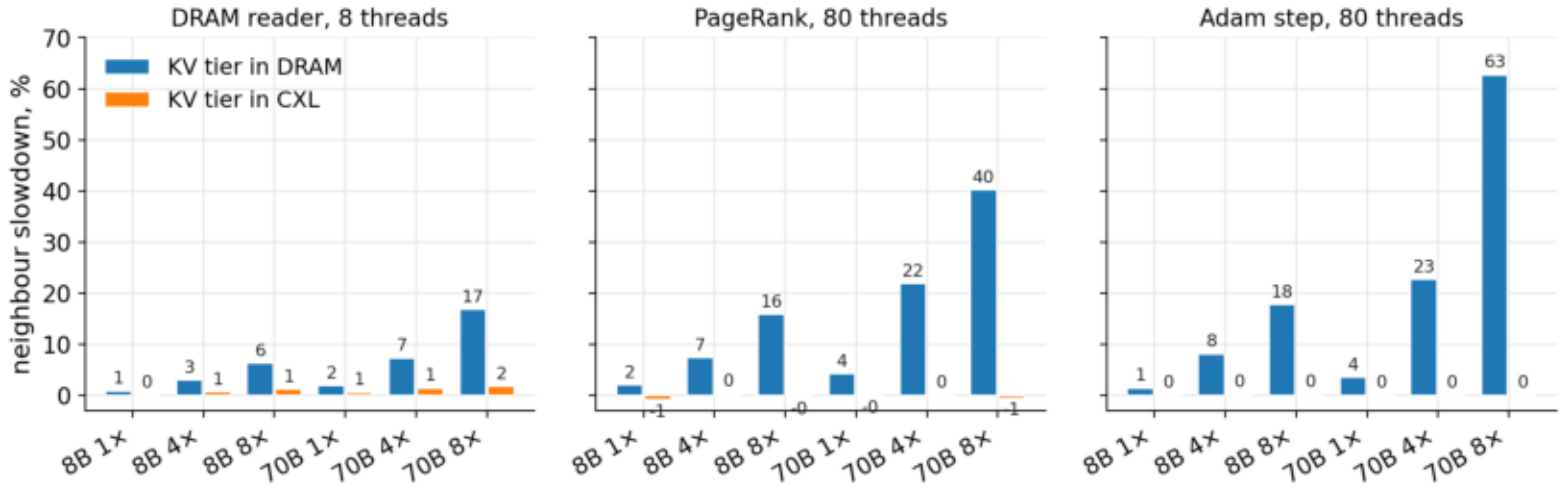
- Trace-driven: request arrivals and block hashes from the Mooncake traces at 1×, 4× and 8× speed; Llama-3.1-8B (64 MiB) and 70B (160 MiB) blocks.
- Stores and loads are CPU copies sized and timed from the real runs: 9.5 GB/s per transfer, 2048-token pieces, 7,300 tokens/s per request.
- Recorded: offered vs achieved bandwidth, 20 ms peaks, backlog and lateness, hit share, neighbour slowdown against its own quiet window.
- Limit: CPU non-temporal stores are not GPU DMA, and the two worker core groups cap DRAM bursts at ~150 GB/s per 20 ms.

Data: benchmarks/memory/kvemu.c, kvemu2.c, run_exp2b.py · 2026-10-10-lmcache-kv-traffic/layer2 · figure f22_emulation_method

Server-scale KV writes into DRAM slow neighbours up to 63%; into CXL, at most 2%

Part IV · KV cache characterization

Trace-driven emulation



Host memory traffic emulation, c28; neighbour on cpus 0–79, its own 60 s window before the traffic as baseline; one run per cell, 2026-10-10/11

- KV tier in DRAM: neighbours lose 1–4% at 8B 1× (5 GB/s mean) and 16–18% at 8B 8× (39 GB/s). At 70B 8× the Adam step loses 63% (p99 +75%).
- KV tier in CXL: ≤ 2% for every neighbour and load; p99 tails ≤ +4%.
- The harm grows with the offered rate and its 20 ms bursts (offered peaks reach 300 GB/s).

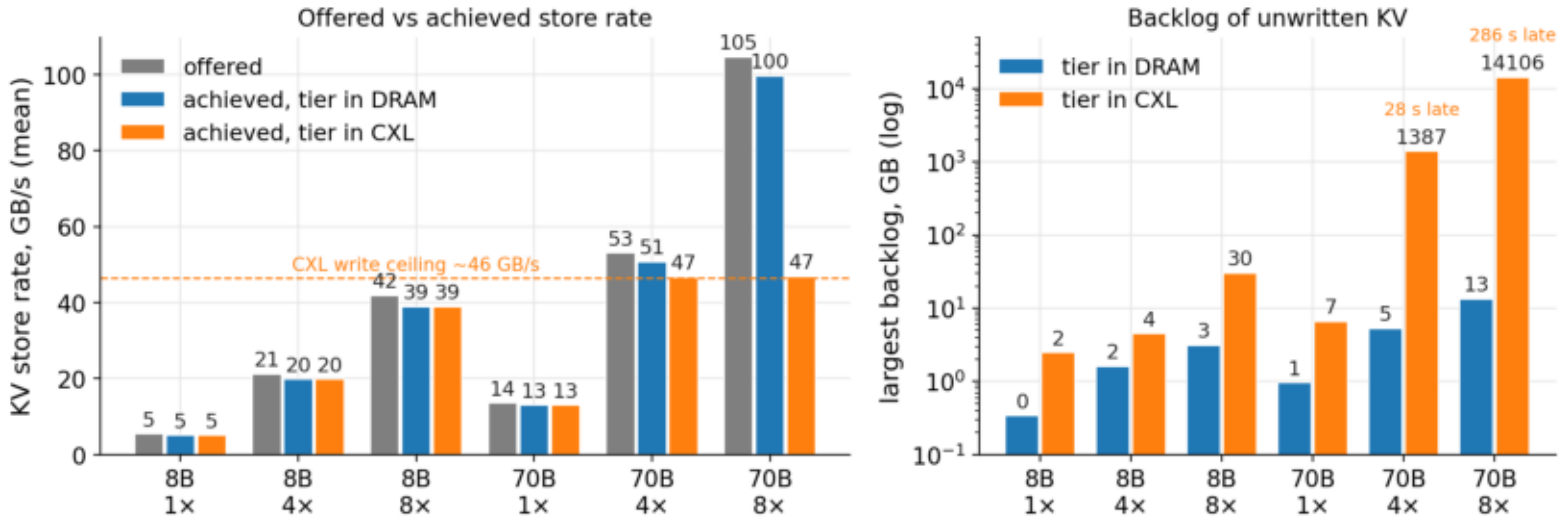
Takeaway: Server-scale KV writes are a real DRAM aggressor. CXL placement removes the harm.

Data: 2026-10-10-lmcache-kv-traffic/layer2/summary.csv (one run per cell, 2026-10-10/11) · figure f10_emulation_neighbors

CXL protects readers but takes at most ~46 GB/s; heavier loads fall behind

Part IV · KV cache characterization

Trace-driven emulation



Host memory traffic emulation (CPU non-temporal stores, not GPU DMA), c28, conversation trace, 200 GiB tier, one run per cell, 2026-10-10/11

- Both tiers keep up to 8B 8× on average, but CXL queues the bursts: 30 GB of backlog, stores 0.6 s late.
- 70B 4× offers 53 GB/s. CXL achieves 47 and falls 1.4 TB behind (28 s late). 70B 8× offers 105 GB/s: 14 TB behind.
- DRAM keeps up at every load, at the cost of the neighbours on the previous slide.

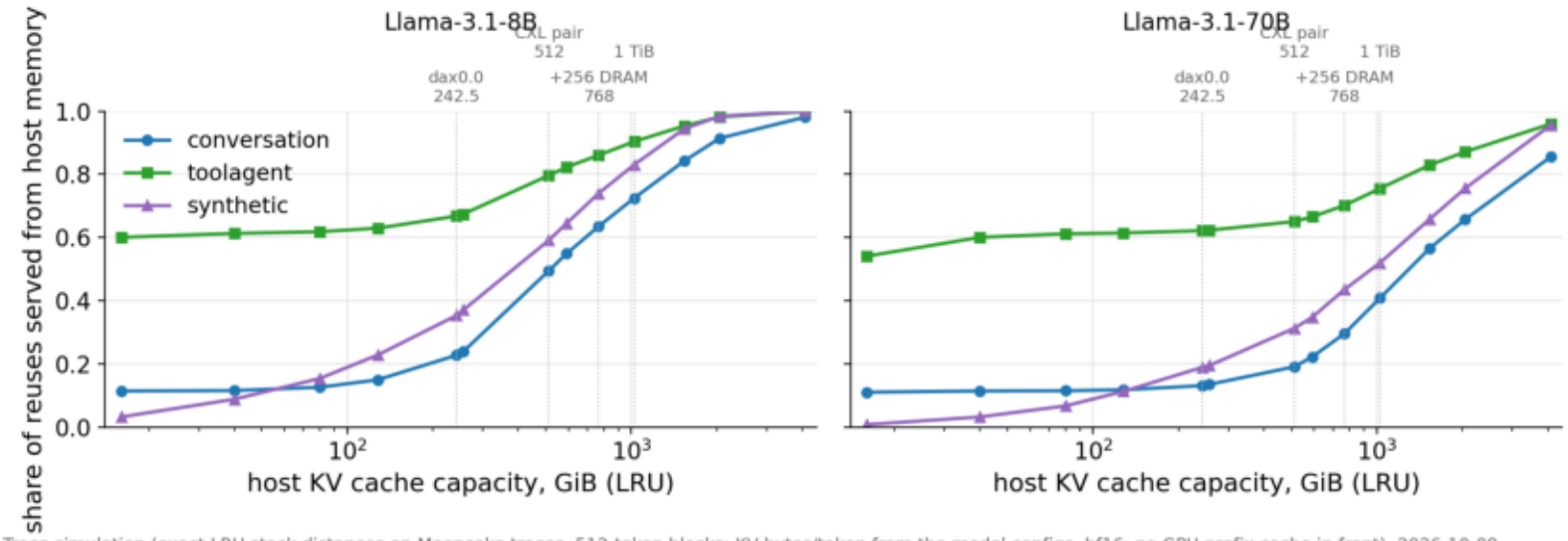
Takeaway: CXL-first alone cannot carry heavy loads. The missing bandwidth must come from DRAM, under control of the neighbours' SLO.

Data: 2026-10-10-lmcache-kv-traffic/layer2/summary.csv · figure f09_emulation_rate

The host KV cache is capacity-bound; DRAM holding other blocks adds +3 to +15 points of hits

Part V · Capacity and memory management

Trace simulation | Estimate



Trace simulation (exact LRU stack distances on Mooncake traces, 512-token blocks; KV bytes/token from the model configs, bf16; no GPU prefix cache in front), 2026-10-09

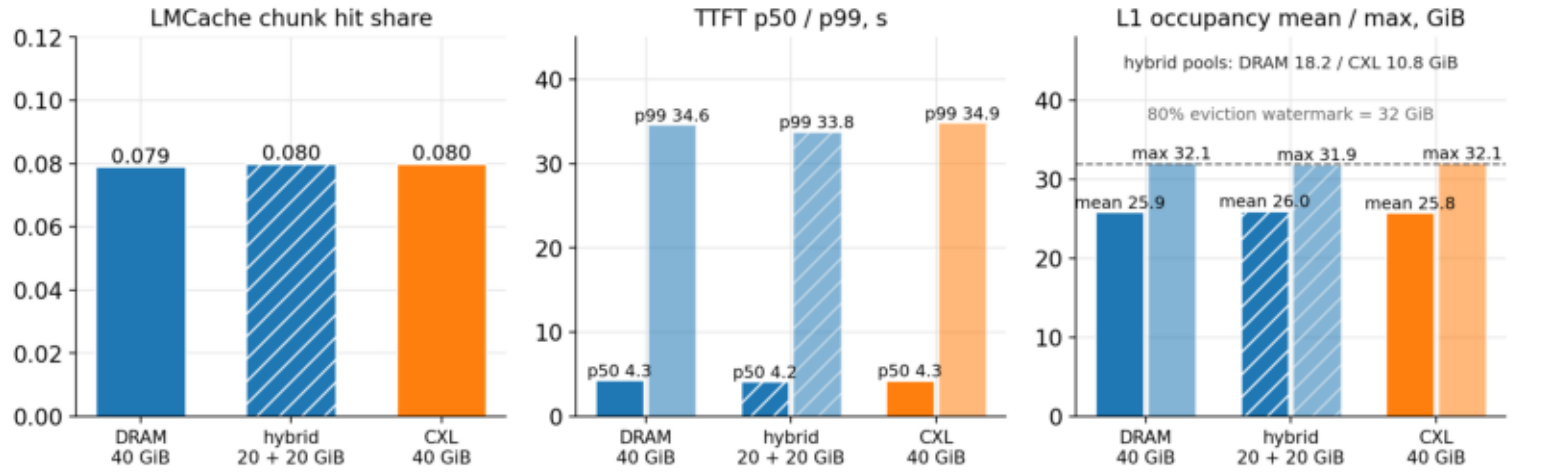
Takeaway: For GPU-read KV, DRAM is worth more as extra capacity than as a faster copy: our GPU loads DRAM and CXL at the same speed.

- One hour of the chat or agent trace creates ~11 TiB of distinct 8B KV. c28 has 512 GiB of CXL.
- LRU, 8B: 512 GiB of CXL serves 49 / 80 / 59% of reuses (conversation / tool agent / synthetic). +256 GiB of DRAM holding other blocks: +14 / +6 / +15 points. 1 TiB: 72 / 90 / 83%.
- 70B: 19 / 65 / 31% at 512 GiB. Copies of CXL blocks in DRAM add nothing to these shares.
- Assumptions: no GPU prefix cache in front, LRU, one host per trace, bf16 KV sizes from the model configurations.

Data: 2026-10-09-kv-capacity-characterization/kv-capacity.csv (exact LRU stack distances) · figure f11_kv_capacity

Real LMCache: DRAM, DRAM-first hybrid and CXL-only L1 serve the same hits and TTFT on this GPU

Real application



Real application: vLLM 0.31 + LMCache 0.5.5 multi-process, Qwen2.5-1.5B, RTX 4060 Ti, c28; conversation trace 0–180 s at 0.08× (556 requests), one run per configuration, 2026-10-12

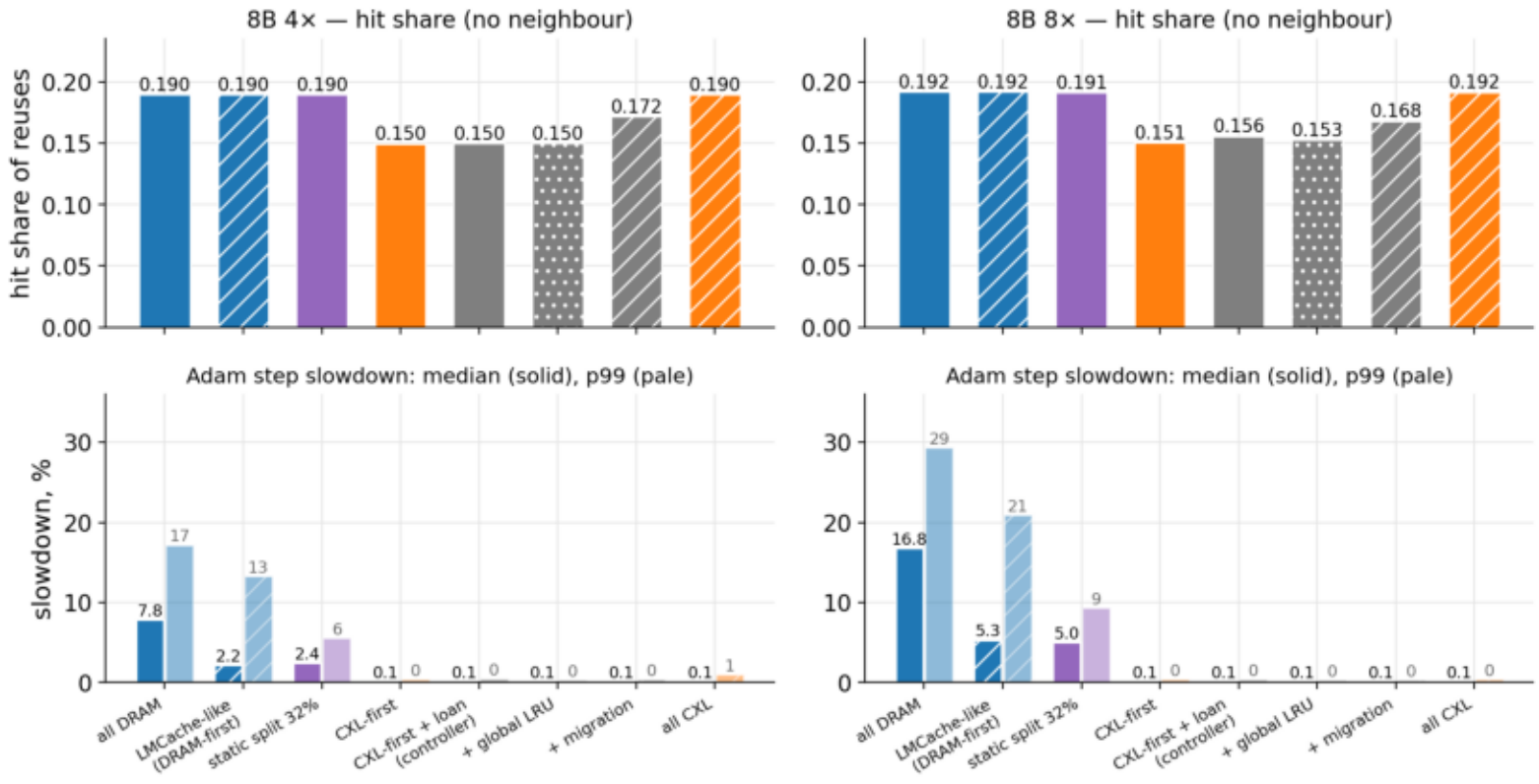
Takeaway: On this GPU the tier does not change serving. Capacity effects need the larger emulated cache.

- Same 40 GiB L1 as pinned DRAM, LMCache's hybrid (20 GiB DRAM + 20 GiB Device-DAX) and pure CXL; 556 requests of the conversation trace at 0.08×.
- Hit share 0.079–0.080; TTFT p50 4.2–4.3 s, p99 34–35 s. Per request, TTFT differs by ≤ 0.04 s (noise ±0.15 s).
- LMCache's 80% eviction watermark leaves 8 GiB unused. In the hybrid that slack sits entirely in the CXL part: DRAM 18.2 / CXL 10.8 GiB used.
- LMCache's hybrid is hard-wired DRAM-first. A CXL-first order needs a code change, so CXL-first and borrowing were emulated.

Data: 2026-10-11-capacity-matched-kv/real/summary.txt (multi-process LMCache, 2026-10-12) · figure f12_real_lmcache

At equal capacity, static admission beats our migration policies; CXL-first alone idles a fifth of the hits

Trace-driven emulation



Host memory traffic emulation (kvemu2), c28, 200 GiB effective capacity for every policy, conversation trace, one run per cell, 2026-10-11/12

- 200 GiB of effective capacity for every policy; same trace, warm-up and eviction.
- Policies that fill both tiers reach hit share 0.190. CXL-first leaves 64 GiB of DRAM idle: 0.150. A global LRU does not help.
- The controller lends nothing next to Adam. Calm-time migration recovers half (0.172) for ~0.2 GB/s of DRAM writes — nothing next to Adam.
- A static 32% split reaches 0.190 at +2.4% median / +5.5% p99 Adam slowdown, inside the 7% SLO.

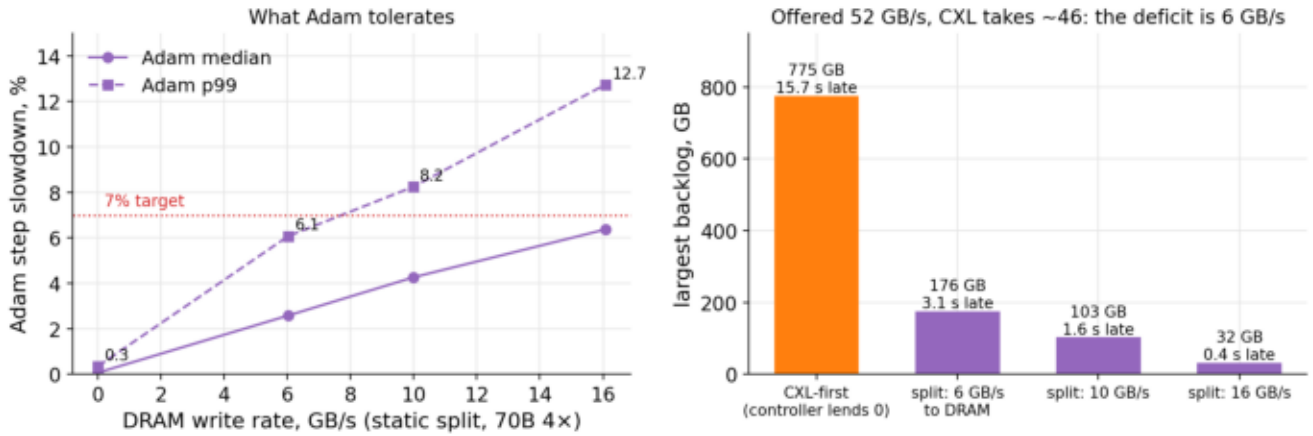
Takeaway: Migration is cheap in DRAM writes but earns no unified manager yet. A plain admission split did better.

Data: 2026-10-11-capacity-matched-kv/emulation/summary.csv (81 runs, 2026-10-11/12) · figure f13_admission_migration

The current controller reads a DRAM-heavy neighbour as 'no headroom' and lends nothing

Part VI · Budget-gated admission

Trace-driven emulation



Host memory traffic emulation, c28, 70B 4× next to the 80-thread Adam step, one run per point, 2026-10-12. The controller's probe reads Adam's own DRAM load as 'busy' (275 ns vs 127 ns idle) and lends nothing

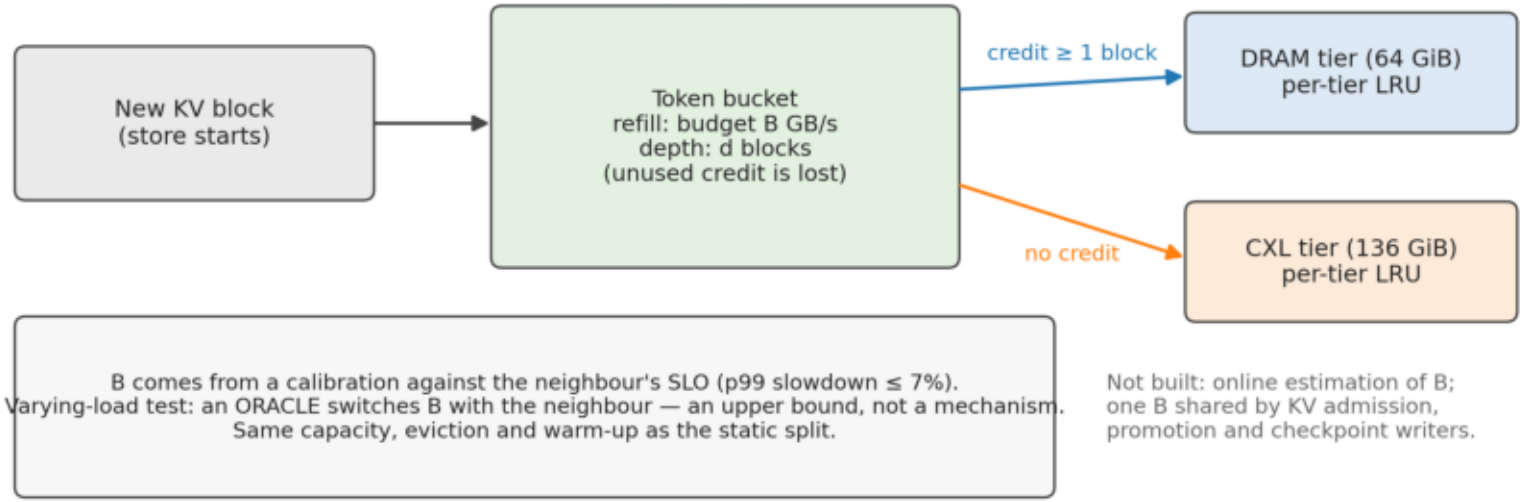
- 70B 4× offers 52 GB/s and CXL takes 46. The deficit is 6 GB/s.
- The Adam step alone raises the DRAM latency probe to 275 ns (idle: 127 ns). The guard treats that as 'busy' and lends 0 GB/s. The backlog grows past 700 GB.
- Static splits show what Adam tolerates: 6 GB/s at +6.1% p99, 10 GB/s at +8.2%, 16 GB/s at +12.7%.

Takeaway: A small, steady DRAM write budget fits the 7% SLO and bounds the backlog. The probe-based guard cannot find it.

Data: 2026-10-11-capacity-matched-kv/emulation (opt-70b-4x-split24 / split40 / split, cxlfirst; daemon logs) · figure f14_controller_limit

Budget-gated admission: a token bucket decides, block by block, whether a new block may use DRAM

Part VI · Budget-gated admission



Takeaway: The simplest form of a shared DRAM write budget. Online estimation of B is not built.

Emulated policy | Proposal (not implemented)

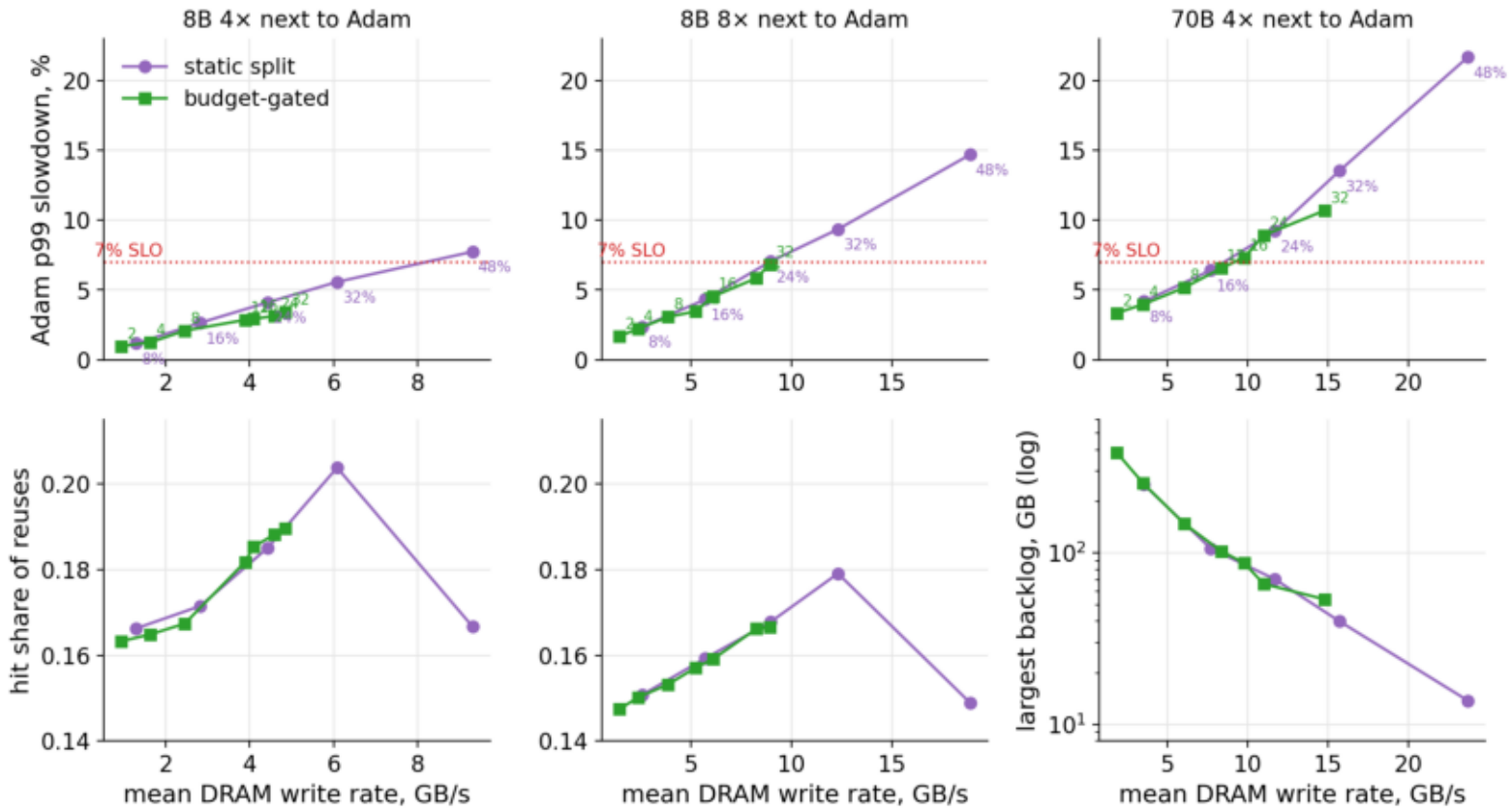
- Credit refills at the budget B GB/s and is capped at d blocks (d = 1 by default). A new block goes to DRAM when a whole block of credit exists; otherwise to CXL.
- Unused credit is lost, so DRAM writes stay smooth. Same capacity (64 + 136 GiB), per-tier LRU and warm-up as the static split.
- B is calibrated on a separate trace segment against the SLO (p99 slowdown ≤ 7%). An oracle that switches B with the neighbour gives an upper bound.

Data: benchmarks/memory/kvemu2.c (policy budget, KV2_BUCKET_BLOCKS), 2026-10-12/13 · figure f15_budget_design

Calibration: smoother DRAM writes buy SLO room, but a one-block bucket caps DRAM's share at ~25%

Part VI · Budget-gated admission

Trace-driven emulation



Host memory traffic emulation, c28, calibration segment (trace 2040 s on), 60 s quiet window, one run per point, 2026-10-12. Labels: split share / budget GB/s

- Sweep next to Adam on a separate trace segment: split 8–48%, budget 2–32 GB/s, one run per point.
- Same mean DRAM rate, lower p99 for the budget (8B 8×: +6.8% vs +7.0% at ~9 GB/s); 20 ms peaks 20 vs 40–50 GB/s.
- A chunk's four blocks fall due together: one takes the bucket, three go to CXL. DRAM's share of writes stays ~25%.
- Hit share peaks when DRAM's share of new blocks equals its 32% share of capacity (32%: 0.204; 48%: 0.167).

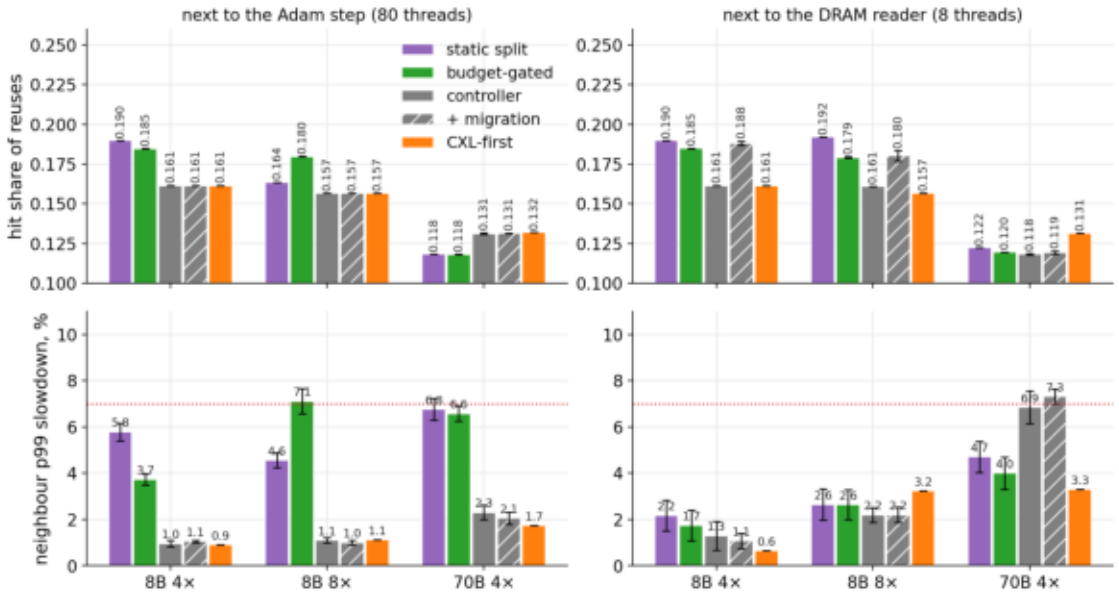
Takeaway: Smoothness buys SLO room; the one-block bucket gives part of it back by under-using DRAM's capacity.

Data: 2026-10-12-budget-gated-admission/calibration/summary.csv (72 runs, trace segment from 2040 s) · figure f16_bga_calibration

Evaluation under the same 7% SLO: budget-gated admission ties the best safe split; both beat the controller

Part VI · Budget-gated admission

Trace-driven emulation



Host memory traffic emulation, c28, evaluation segment (trace 600 s on), settings from the independent calibration, 2026-10-12/13. Bars = mean of 1–3 independent runs per cell, error bars = half the range; 70B 4× hit shares not comparable (overload)

Takeaway: Inside the same SLO a bandwidth budget ties the best static split: smoother writes, no extra hits. Both beat the current controller beside a DRAM-heavy neighbour.

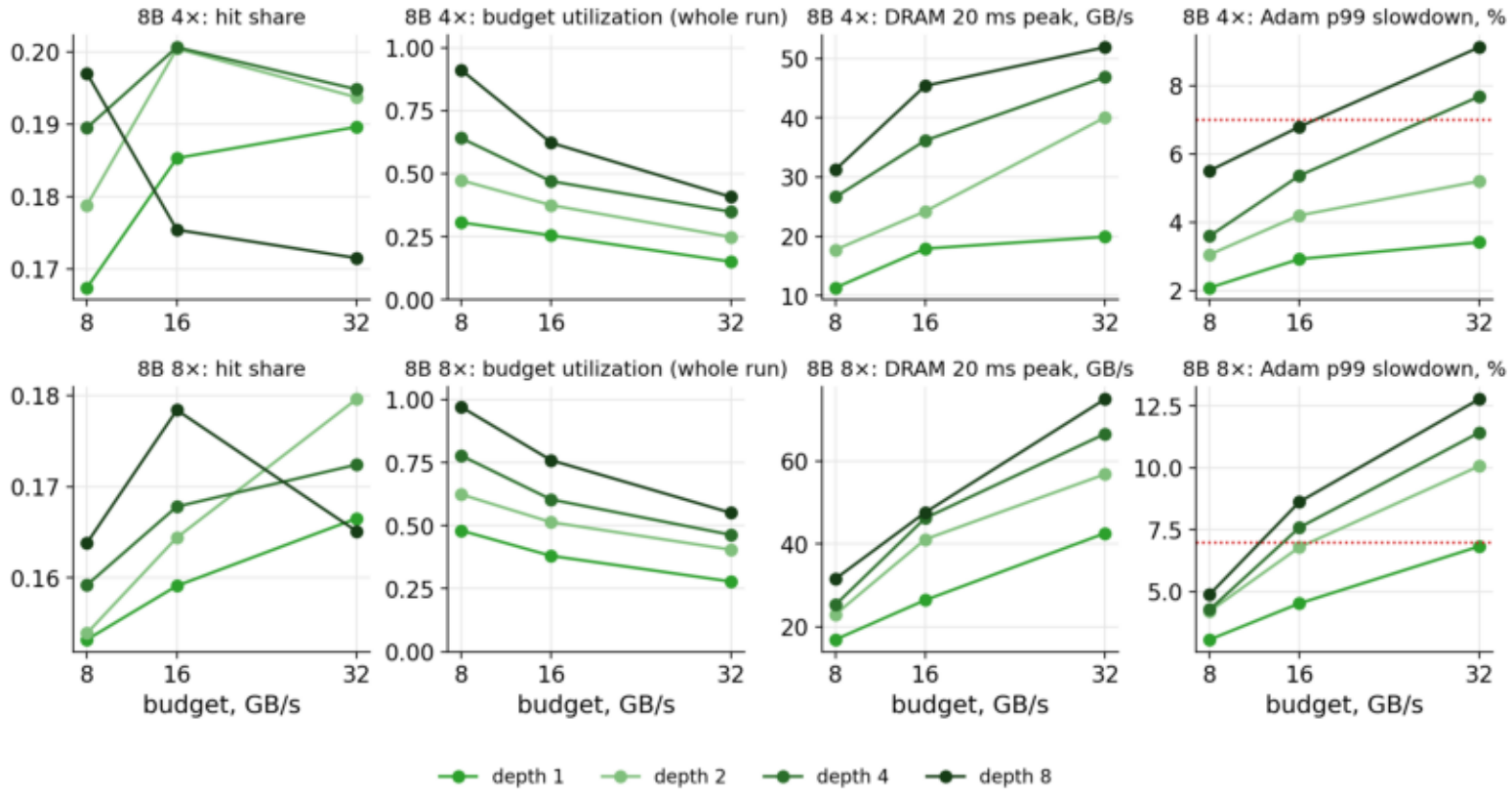
- Evaluation segment; settings fixed in advance by the calibration rule.
- Next to Adam: 8B 4× split 0.190 at +5.8%, budget 0.185 at +3.7%. 8B 8× budget 0.180 at +7.1% [6.6..7.6] — on the SLO — vs split 0.164 at +4.6%; held to 7%, a 24 GB/s budget ties the split. 70B 4×: a tie.
- The controller lends nothing next to Adam (0.157–0.161). Migration matches the split next to the reader at 0.2–1.9 GB/s of DRAM writes but breaks the SLO at 70B 4×.
- 3 runs per cell: hit shares reproduce to ±0.001, the p99 to ±0.5 points.

Data: 2026-10-12-budget-gated-admission/evaluation/summary.csv (2026-10-12/13) · figure f17_bga_evaluation

A deeper bucket uses the budget fully, but on the evaluation segment it earns no extra hits

Part VI · Budget-gated admission

Trace-driven emulation



Host memory traffic emulation, c28, calibration segment next to the Adam step, one run per point (depth 1 = calibration runs), 2026-10-12/13

- Sweep next to Adam, calibration segment: depth 1, 2, 4, 8 × budget 8, 16, 32 GB/s. At 8 GB/s the utilization rises from 0.31 to 0.91; 20 ms peaks grow from 20 to 31–52 GB/s.
- The hit share follows the admitted share, not the depth: it peaks when DRAM gets ~32% of the new blocks (0.201 at depth 4 / 16 GB/s; the 32% split: 0.204). Adam's p99 follows the mean DRAM rate.
- Evaluation of depth 4 / 16 GB/s (3 runs): 0.187 at +5.3% — +0.002 over depth 1 for +1.6 points of p99, below the split's 0.190.

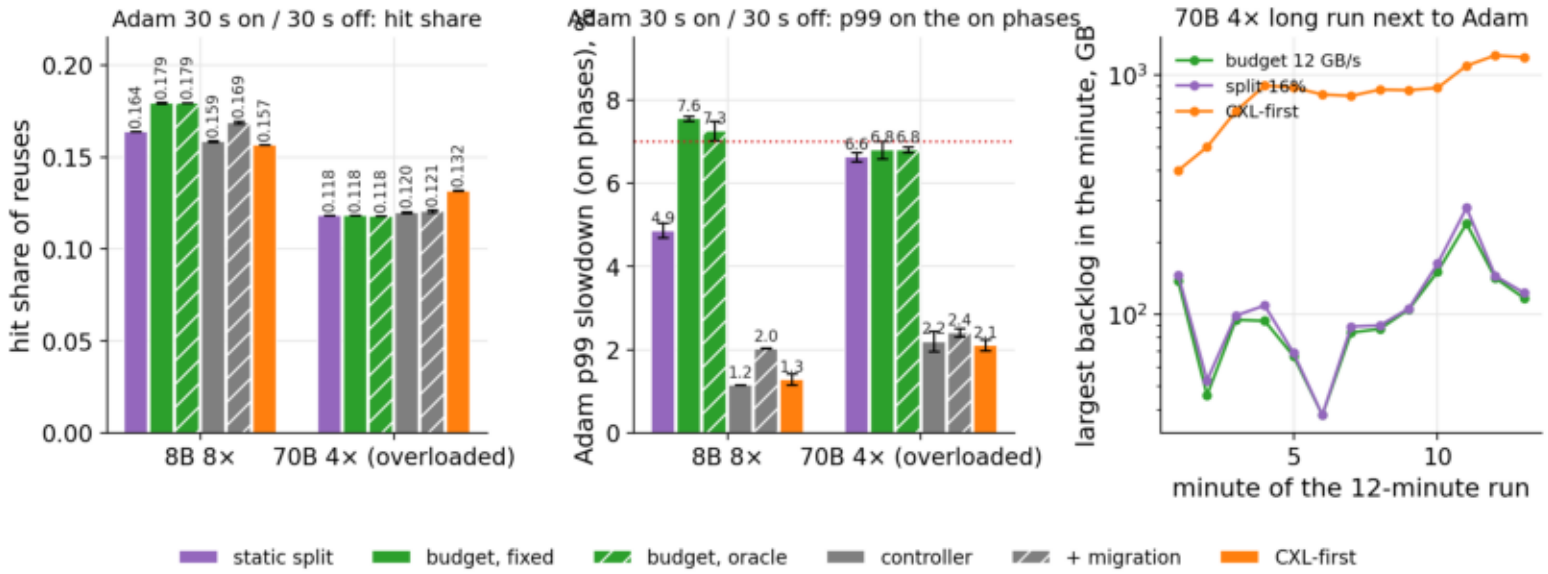
Takeaway: With the right depth the budget policy is a smoother way to express the same static split, not a way to beat it.

Data: 2026-10-12-budget-gated-admission/depth (18 runs) and depth-eval (3 runs), 2026-10-13 · figure f18_bga_depth

Adapting the budget to the neighbour drains the backlog; 70B 4× next to Adam stays marginal

Part VI · Budget-gated admission

Trace-driven emulation



Host memory traffic emulation, c28; left/middle: Adam 30 s on / 30 s off, 2 runs per cell (error bars = half range); right: 720 s of traffic, one run each; 2026-10-13

Takeaway: Dynamic adaptation buys backlog, not hits. A steady 70B 4× load next to Adam exceeds CXL plus the safe DRAM budget: capacity is insufficient, not merely tight.

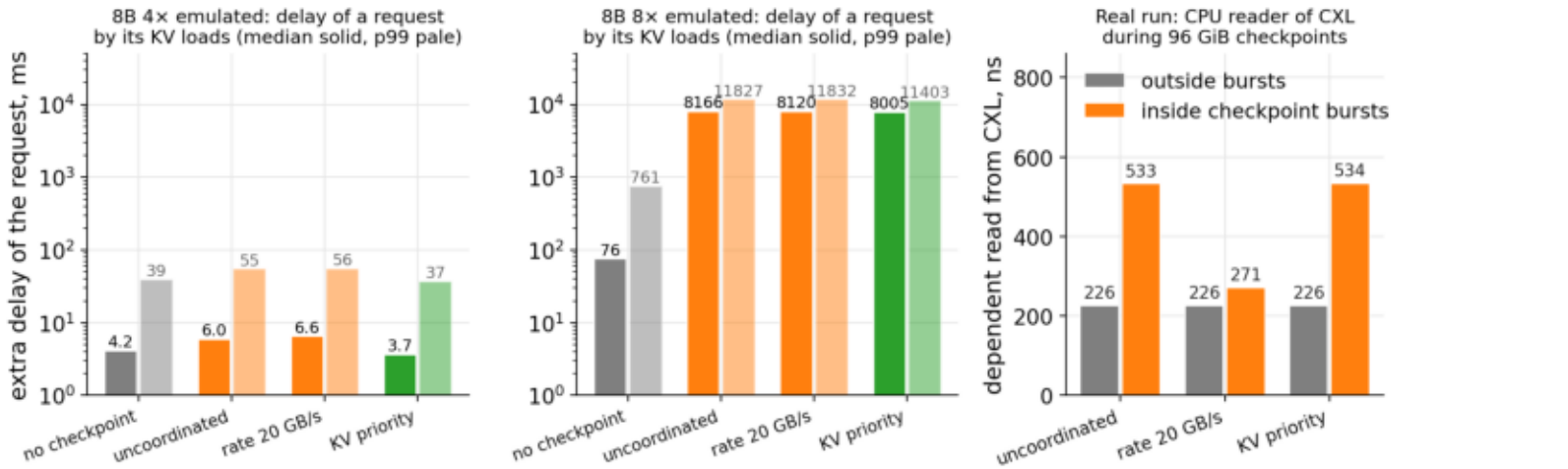
- Adam runs 30 s, pauses 30 s. At 70B 4× an oracle budget (12 GB/s while Adam runs, 32 while it pauses) cuts the peak backlog from 137–145 GB to 60 GB at the same Adam p99 (+6.8%). The controller lends only in the pauses: 201 GB.
- At 8B 8× the oracle equals the fixed budget (32 GB/s is already the largest swept); its +7.6% confirms 32 GB/s is above the SLO next to Adam.
- Long run, 12 min of 70B 4× next to Adam: stores reach 51.5 GB/s against 54 offered; the backlog swings between 38 and 280 GB; Adam's p99 sits at the SLO (budget +6.9%, split +7.3%). CXL-first grows to 1.2 TB.

Data: 2026-10-12-budget-gated-admission/varying (24 runs, 2 per cell) and longrun (3 runs), 2026-10-13 · figure f19_bga_varying

Checkpoint writes and KV reads on the same CXL cards: KV priority protects loads, not the shared write pipe

Part VII · Mixed applications

Real application | Trace-driven emulation



Left/middle: host memory traffic emulation (kvemu3, toolagent trace, KV tier 150 GiB in CXL, 64 GiB checkpoint every 8 s), one run per cell. Right: real vLLM + LMCache run on c28, 2026-10-12

Takeaway: Two writers on one CXL pipe need admission across writers: lend DRAM to one, or pace the checkpoint below the residual CXL capacity. Not built.

- Real run: 96 GiB checkpoints every 8 s into the cards that hold the KV cache. KV loads and TTFT do not change (the GPU link is the limit). A CPU reader of CXL: 226 → 533 ns inside the bursts.
- Emulated 8B 4×: uncoordinated checkpoints add 44% to the median request delay. KV priority removes it (+14% checkpoint time). A 20 GB/s rate limit does not.
- Emulated 8B 8×: KV stores already use ~36 of ~46 GB/s. Any checkpoint pushes the median delay from 76 ms to ~8 s in every mode — confirmed from the raw load queues.

Data: 2026-10-12-checkpoint-kv-mixed (real: 4 runs, tool agent trace at 0.08×; emulation: kvemu3, 8 runs) · figure f20_ckpt_kv

What we have learned

Part VIII · Current conclusions

Summary

1 DRAM bulk writes can slow neighbours severely

A 4-thread reader keeps 8% under saturating writers; server-scale KV writes cost neighbours up to 63%. (Slides 2, 12)

2 CXL writes remove the harm under a placement condition

≤ 2% for every neighbour when the writer's threads sit on an unrelated core group; the paired or own group gives no protection. (Slides 7, 12)

3 CXL write bandwidth limits high-rate workloads

~46 GB/s on the two cards; 70B 4× falls 1.4 TB behind; two writers on one pipe push KV load queueing to ~8 s. (Slides 13, 23)

4 DRAM provides useful extra KV capacity

+3 to +15 points of reuse hits in the trace simulation; idle DRAM costs a fifth of the hits in emulation. (Slides 14, 16)

5 Static admission can outperform current migration policies

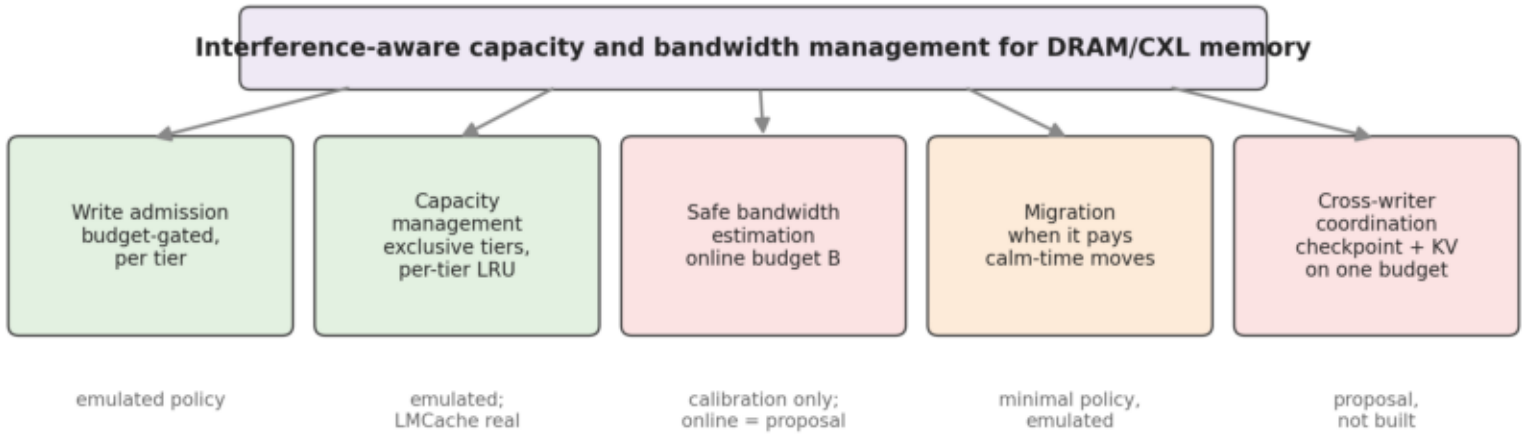
A 32% split beat every controller policy inside the SLO; a bandwidth budget ties it at any bucket depth; only an adaptive budget adds value, by halving the 70B backlog. (Slides 16, 20–22)

6 DRAM budget estimation and cross-writer coordination remain open

The guard lends nothing beside DRAM-heavy neighbours; KV priority cannot share the write pipe. (Slides 17, 23)

Proposed direction: interference-aware capacity and bandwidth management for DRAM/CXL memory

Part VIII · Current conclusions



Two budgets: DRAM write bandwidth (one shared budget) and DRAM capacity. CXL stays the default home of bulk writes.

Takeaway: A minimal manager is justified only where the data show a gain: admission with a budget first, migration where DRAM is calm.

Proposal (not implemented)

- Keeps the original thesis: bulk writes go to CXL by default; DRAM is granted on evidence.
- Adds the advisor's capacity view: each block lives in one tier; DRAM capacity is a second budget.
- Implemented so far, as emulated policies: budget-gated admission, per-tier capacity management, a minimal migration rule; and the real LMCache integration for DRAM, hybrid and CXL tiers.
- Proposals only: online estimation of the safe DRAM budget; one budget shared across KV admission, promotion and checkpoint writers.

Data: This deck; docs/experiments/INDEX.md · figure f23_direction

Next steps and open questions

Part VIII · Current conclusions

Proposal (not implemented)

1 Can dynamic admission beat the best static split?

For a fixed budget: no, it ties the split at any depth (slides 20–21). It pays only by adapting to the neighbour (slide 22), which needs an online estimate of the safe budget.

2 Can we estimate the safe DRAM bandwidth online?

The probe-based guard fails beside DRAM-heavy neighbours. Candidate: a write-rate probe calibrated against the measured SLO curve.

3 When are promotion or demotion necessary?

So far only next to calm DRAM, and never better than admission. Decide after the repeats; keep migration minimal.

4 How should checkpoint writes and KV reads share CXL bandwidth?

Cross-writer admission on one budget; pace the checkpoint below the residual CXL capacity; protect latency-sensitive CXL readers.

5 What minimal unified manager do the data justify?

Admission with a budget, per-tier capacity management, calm-time migration. Nothing more until it is measured.

6 Priorities for the next phase

1. Online estimation of the safe DRAM write budget (the current guard finds none beside Adam). 2. Cross-writer admission prototype for checkpoint + KV.
3. A selection rule with an SLO margin.
4. CXL-first patch for real LMCache.